Challenge Brief: Traceable Harmonization of Ocean Data for ML Readiness

Context

Modern ocean science and environmental monitoring rely on large volumes of heterogeneous data produced by different sensors, platforms, institutions, and missions. While many of these datasets are available in common formats such as NetCDF, their effective use for machine learning (ML) remains challenging due to differences in variable naming, units, reference frames, metadata quality, and implicit assumptions. In addition to technical heterogeneity, increasing attention is being given to data provenance, traceability, integrity, and sovereignty. As ocean datasets move through processing pipelines and are reused across institutions and applications, it becomes important to ensure that the origin, transformation history, and authenticity of each data element can be verified. This challenge asks you to design and implement the first layer of an ocean data pipeline: from immutable source files to an ML-ready Zarr data store. The focus is not on building models, but on making heterogeneous data comparable, traceable, secure, and reusable for downstream ML workflows.

Problem Statement

You are provided with several heterogeneous ocean observation datasets in NetCDF format. The datasets partially overlap in space and time but differ in:

- variable names and semantics
- units and conventions
- spatial and temporal reference frames
- metadata completeness and quality

Your task is to design and prototype a pipeline that ingests these source datasets and produces a harmonized Zarr store suitable for ML experimentation. In addition to harmonization, the system should make it possible to trace the lineage of a data point or derived variable back to its original source file and transformation steps, ensuring transparency and auditability of the processing pipeline.

Constraints (Design Invariants)

- **1. Source immutability**

  Source NetCDF files must be treated as immutable. The pipeline must not modify source data in place.

- **2. Re-derivability**

  The same source datasets may need to be reprocessed later to produce a different ML-ready representation without duplicating or altering the raw sources.

- **3. Traceability through aggregation**

  When data from multiple source files are aggregated into a single dataset, it must be possible to determine which source files and transformations contributed to each derived variable or data element.

- **4. Explicit transformations**

  All transformations between source data and output must be explicit, inspectable, and reproducible. Black-box preprocessing is not acceptable.

- **5. ML usability without domain hardcoding**

  The resulting dataset should be usable by an ML practitioner without deep knowledge of the original instruments while preserving enough information for domain scientists to audit assumptions.

- **6. Data integrity and security awareness**

  The architecture should consider mechanisms that help ensure the integrity and authenticity of data throughout the pipeline (for example cryptographic hashing, audit logs, or tamper evident metadata).

- **7. Data provenance and sovereignty**

  The system design should consider how the provenance and ownership context of datasets can be preserved as they move through the pipeline, ensuring that data remains uniquely identifiable and attributable.

Expected Outcomes

- Propose a clear architectural approach for ingestion and harmonization
- Implement a minimal working prototype demonstrating the approach
- Explain how traceability, integrity, and reprocessing are supported
- Discuss trade-offs and limitations of the design

There is no single prescribed solution. The emphasis is on the abstractions and design choices used to manage semantic complexity, traceability, and trustworthy data handling.

Guiding Question

How would you design the system so that changing the interpretation of a variable (for example, its meaning, unit normalization, or aggregation rule) does not require rewriting the ingestion code or modifying the original source data?

Additionally, how can the architecture ensure that the provenance and integrity of each derived dataset can be verified as it moves through the processing pipeline?

Evaluation Criteria

- Clarity and robustness of the architecture
- Degree of traceability, provenance, and reproducibility
- Separation between data ingestion and interpretation
- Suitability of the resulting Zarr store for ML workflows
- Consideration of data integrity and security aspects
- Quality of reasoning and documentation